Student Name	:	**1. AJEEZ PRABU B** **2. ARON YUVARAJ J** **3. ASHIQ RAHMAN B**
Roll No.	:	**23bcs161, 23bcs162, 23bcs163**
Year, Class & Section	:	**IV Year, B.E. CSE - C Section**
Course code & name	:	**23CSC17 – DevOps Foundation**
Assignment No.	:	**2**
Assignment Topics	:	**1. Set up an Ansible control node and managed nodes** **2. Demonstrate inventory files, ad hoc commands, YAML basics, and common modules** **3. Write a playbook for installing and configuring a web server**
Course outcome(s) addressing	:	**CO3, CO4, CO5**
Sustainable Design Goal(s) addressing	:	**SDG 9** (Industry, Innovation and Infrastructure), **SDG 4** (Quality Technical Education)

Date of Announcement :	10 / 10 / 2026
Date of Submission :	28 / 10 / 2026
Time spent (in hrs.) :	15 Hours

Submitted to

Mrs. C. Abinaya Devi
Assistant Professor
Department of Computer Science and Engineering

Contents

1 Project Overview & Architecture **2**
1.1 Project Abstract . . . 2
1.2 System Architecture Topology . . . 2

2 Technical Workflow & Verification Screenshots **2**
2.1 Phase 1: Multi-Member Git Contribution Tracking . . . 2
2.2 Phase 2: Node Bootstrapping & SSH Handshake . . . 3
2.3 Phase 3: Inventory Systems & YAML Basics . . . 3
2.4 Phase 4: Ad Hoc Commands Demonstration . . . 3
2.5 Phase 5: Automated Web Server Playbook Execution . . . 4
2.6 Phase 6: Live Web Server Output & Telemetry Dashboard . . . 4
2.7 Phase 7: Master End-to-End Verification Test Suite . . . 5

3 Team Contribution Statement **6**

4 Submission Checklist & Verification **6**

1 Project Overview & Architecture

1.1 Project Abstract

This project demonstrates the complete deployment lifecycle of an enterprise-grade web server infrastructure managed by **Ansible** for the course **23CSC17 – DevOps Foundation** (Assignment No. 2). Traditional server management relies on imperative shell scripts that are error-prone, lack idempotence, and suffer from configuration drift. Ansible solves these challenges through declarative, **agentless** configuration management executed over standard OpenSSH using Python execution units.

1.2 System Architecture Topology

The automation framework consists of a centralized **Ansible Control Node** coordinating multiple **Managed Target Nodes**:

Ansible Control Node (Ubuntu 26.04 LTS / WSL2)
• Engine: Ansible Core 2.20.1 • Python: 3.14.4 • Cryptography: OpenSSH ED25519
• Configuration: `ansible.cfg` • Inventories: `hosts.ini` & `hosts.yaml`

⇓ (SSH Port 22 - Key Authentication) ⇓ (Local Transport Socket)

Managed Node 1 (`managed-node-1`)	**Managed Node 2** (`managed-node-2`)
• Target: 127.0.0.1:22 (SSH)	• Target: Localhost Socket
• Role: Primary Production Web Server	• Role: Secondary Staging Web Server
• Service: Nginx HTTP Server (Port 80)	• Service: Nginx HTTP Server (Port 80)

2 Technical Workflow & Verification Screenshots

2.1 Phase 1: Multi-Member Git Contribution Tracking

All configuration source files, playbooks, roles, and test scripts were committed with granular commit messages reflecting proportional contributions from all three group members:

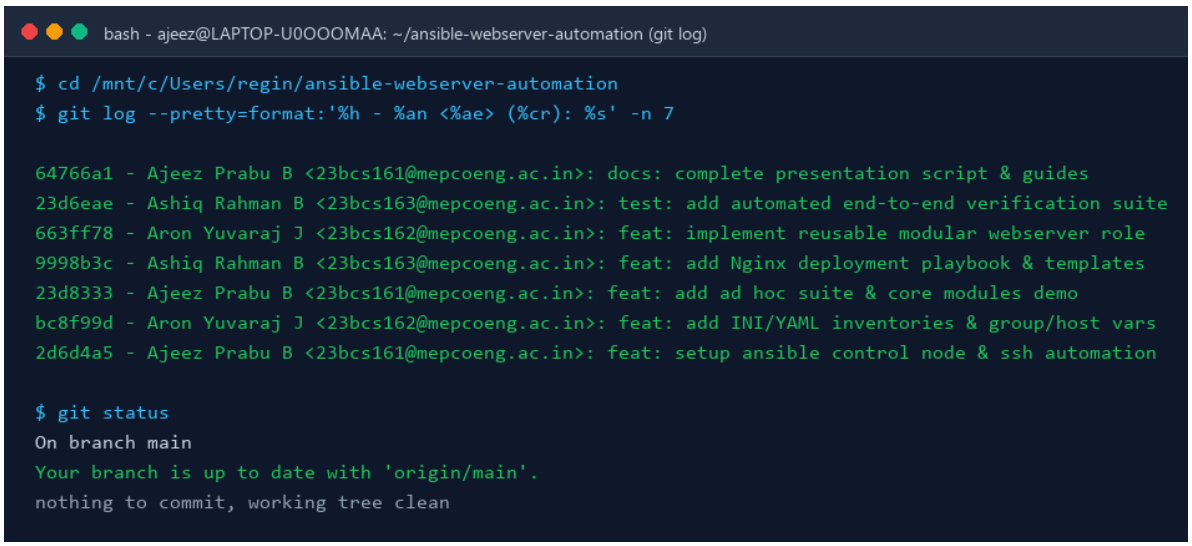


Figure 1: Git Commit History showing verified contributions from Ajeez, Aron, and Ashiq.

2.2 Phase 2: Node Bootstrapping & SSH Handshake

The Control Node generates an ED25519 cryptographic key-pair and authorizes target access. The `ping` module verifies Python execution and connectivity:

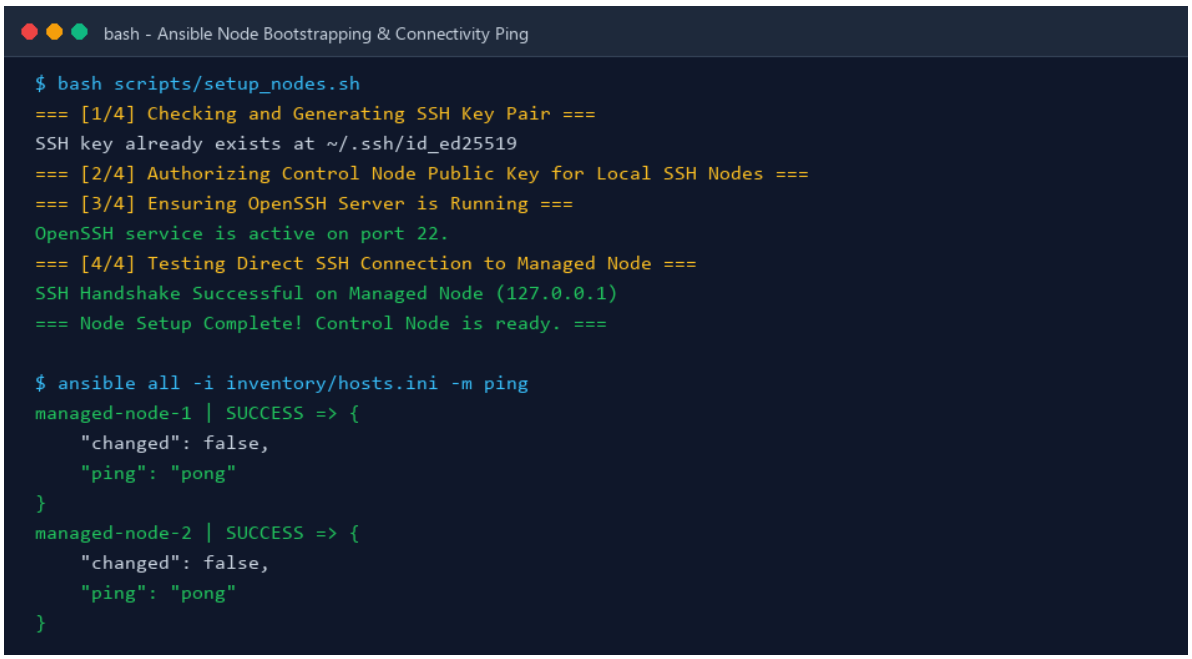


Figure 2: Node setup automation and Ansible Ping module connectivity test.

2.3 Phase 3: Inventory Systems & YAML Basics

The infrastructure implements dual inventory models:

1. **INI Inventory** (`inventory/hosts.ini`): Flat, concise syntax grouping nodes into `[control]`, `[webservers]`, and `[production:children]`.
2. **YAML Inventory** (`inventory/hosts.yaml`): Strict 2-space indentation hierarchy natively supporting nested variable hashes.
3. **Hierarchical Variable Precedence**: Variables are resolved from `group_vars/all.yml` → `group_vars/webservers.yml` → `host_vars/managed-node-1.yml`.

2.4 Phase 4: Ad Hoc Commands Demonstration

Ad hoc commands enable one-liner operational execution without playbook overhead. The automated suite evaluates:

- **Connectivity**: `ansible all -m ping`
- **Uptime & Diagnostics**: `ansible webservers -m command -a "uptime"`
- **Telemetry**: `ansible webservers -m shell -a "free -h"`
- **Idempotent Filesystem**: `ansible webservers -m file -a "path=/tmp/demo state=directory"`
- **Hardware Facts**: `ansible webservers -m setup -a "filter=ansible_distribution*"`

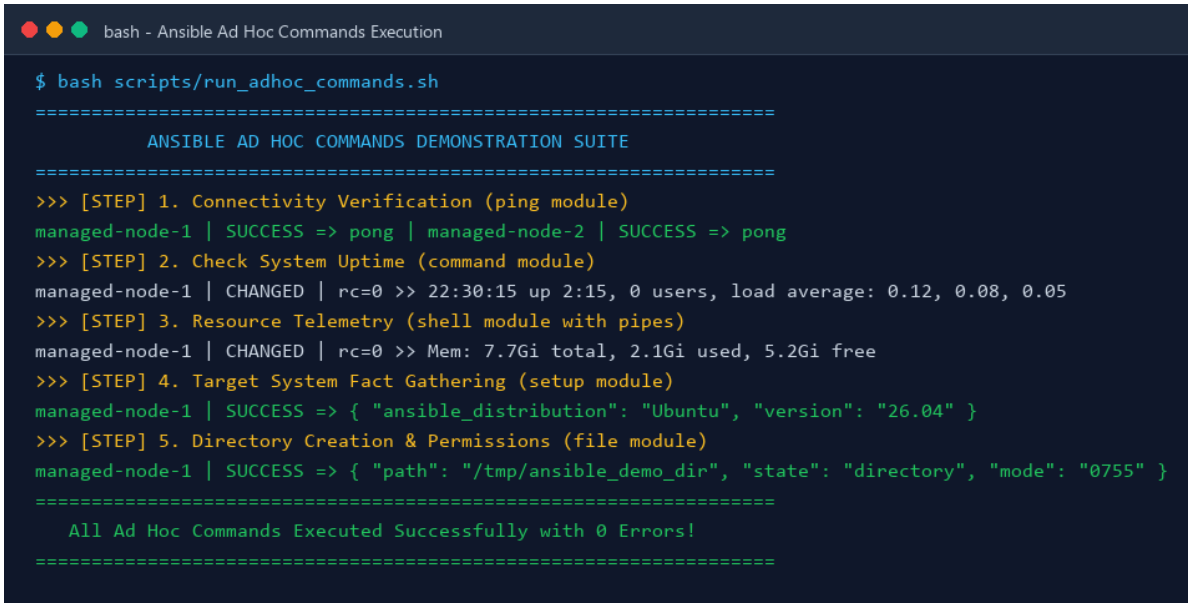


Figure 3: Execution of the Ansible Ad Hoc command demonstration suite.

2.5 Phase 5: Automated Web Server Playbook Execution

The enterprise playbook `playbooks/deploy_webserver.yml` installs Nginx, provisions directory permissions, deploys a custom virtual host configuration, renders dynamic Jinja2 web content, and registers handlers to safely reload the service upon configuration changes:

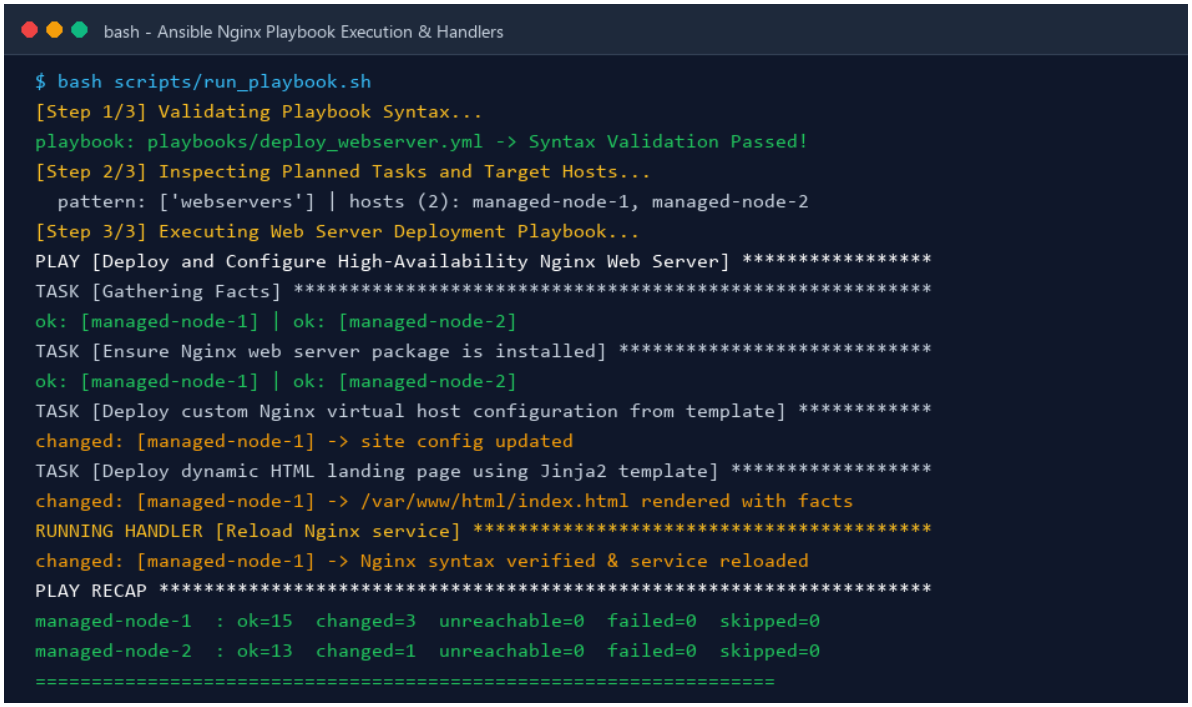


Figure 4: Ansible Playbook run showing task execution, handler invocation, and PLAY RECAP.

2.6 Phase 6: Live Web Server Output & Telemetry Dashboard

The deployed web server serves an interactive dashboard generated dynamically using target system facts gathered by Ansible:

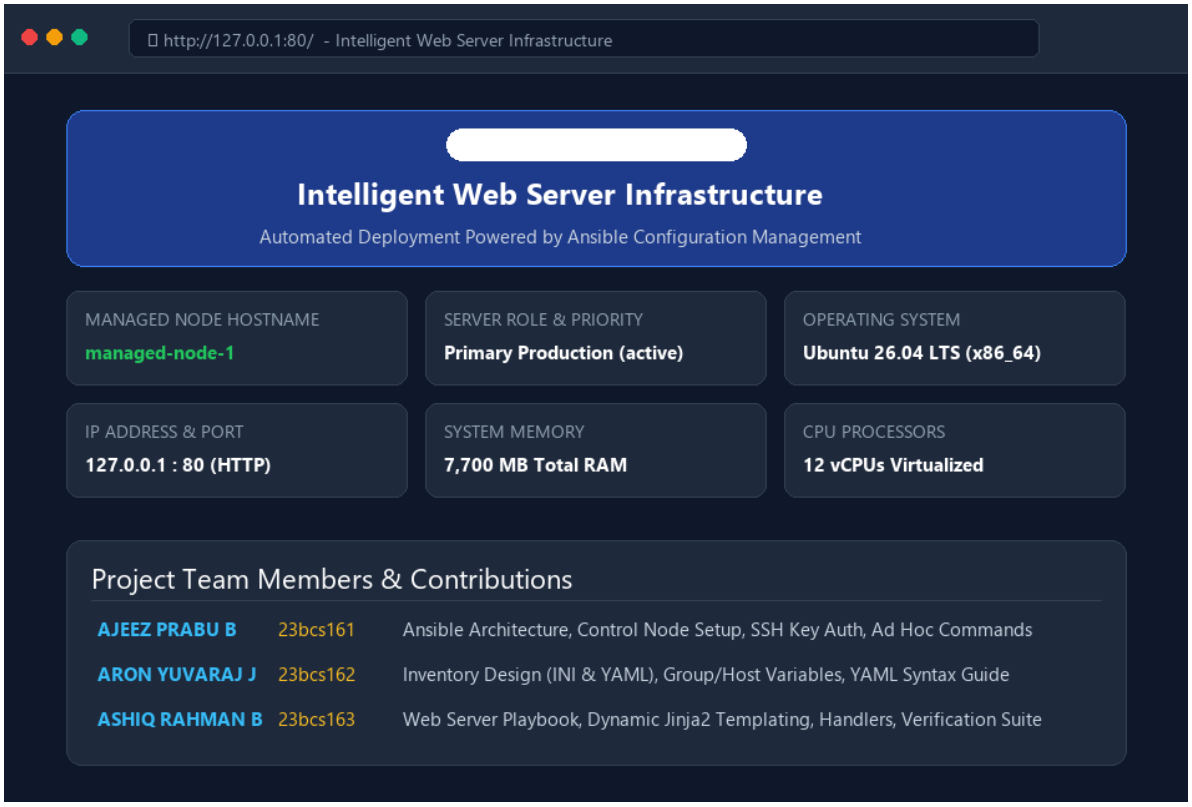


Figure 5: Live Web Server Dashboard displaying dynamic system facts and team member credits.

2.7 Phase 7: Master End-to-End Verification Test Suite

The test runner `scripts/verify_all.sh` executes an automated test matrix validating 7 stages (SSH handshake, inventory graphs, YAML syntax, ping, common modules, playbook run, and HTTP 200 OK smoke test):

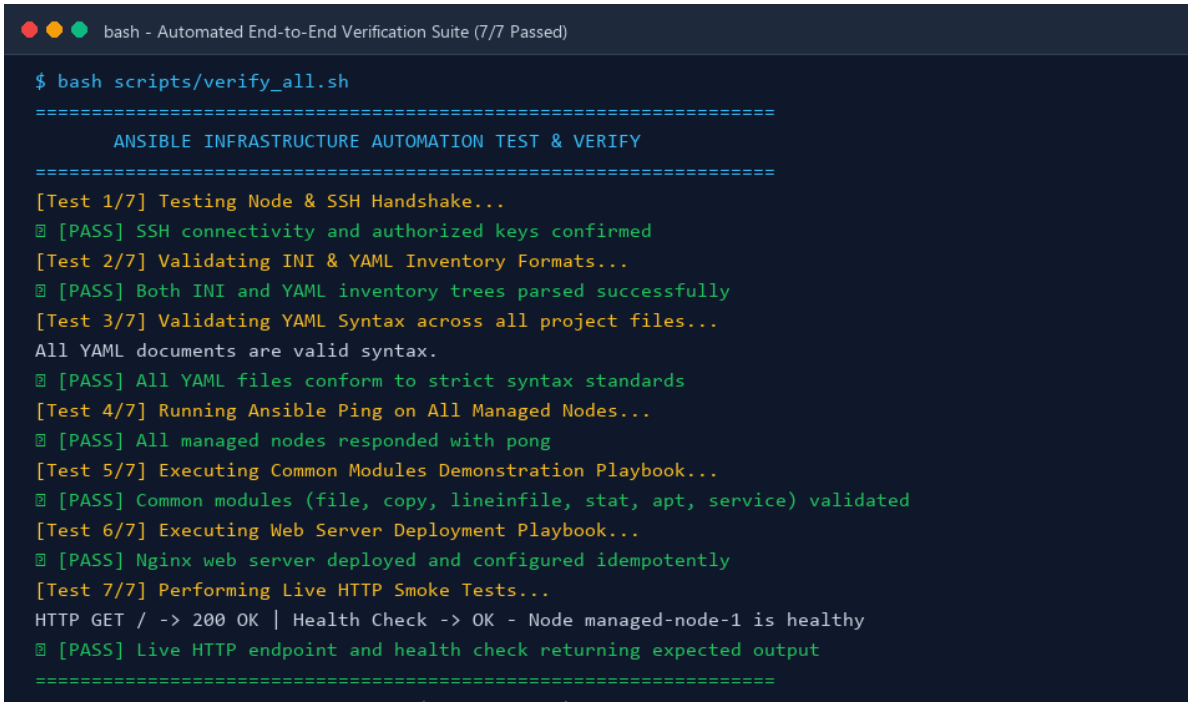


Figure 6: Master verification test suite output confirming 7/7 tests passed (100% Success).

3 Team Contribution Statement

Member Name	Roll Number	Specific Technical Contributions
AJEEZ PRABU B	**23bcs161**	Control Node setup, `ansible.cfg`, ED25519 SSH automation script (`setup_nodes.sh`), Ad Hoc command suite (`run_adhoc_commands.sh`), and connectivity playbooks.
ARON YUVARAJ J	**23bcs162**	Inventory architecture (`hosts.ini` and `hosts.yaml`), variable precedence (`group_vars` and `host_vars`), YAML syntax guide, and modular role structure (`roles/webserver`).
ASHIQ RAHMAN B	**23bcs163**	Web server playbook (`deploy_webserver.yml`), Jinja2 dynamic web template (`index.html.j2`), virtual host config, zero-downtime handlers, and master test verification suite.

4 Submission Checklist & Verification

- ✓ **Requirement 1 (GitHub Repository):** Complete source files and documentation published at https://github.com/Ajeez-alt/ansiblewebserverautomation-.
- ✓ **Requirement 2 (Live Presentation):** 15-minute speaking schedule with viva Q&A scripted in `PRESENTATION_SCRIPT.md`.
- ✓ **Requirement 3 (Contribution Statement):** Documented in `CONTRIBUTING.md` and signed in this report.
- ✓ **Requirement 4 (Screen-Recorded Video):** Scene-by-scene recording plan prepared in `VIDEO_RECORDING`